

Technical Contributions Summary

Dr. Ray Madachy has made broad, sustained contributions to systems engineering distinguished by his work in modeling and simulation, particularly in system cost and process modeling, and development of supporting practical tools. His work exemplifies systems thinking through the application of system dynamics modeling to systems and software engineering processes, as well as other system types. He has also developed comprehensive tools for system dynamics and discrete event modeling implemented in the open-source Systems Engineering Library (se-lib).

His pioneering book *Software Process Dynamics* [1] advanced the use of system dynamics modeling to simulate complex software development processes. His book *What Every Engineer Should Know About Modeling and Simulation* [2] introduces M&S fundamentals and applications to engineers across disciplines.

He has made significant contributions to system cost modeling and tools, including serving as a core member of the COCOMO II [3] development team. His original work includes development of the Expert COCOMO [4] and Expert COSYSMO [13] models/tools, system product line cost/investment modeling with COPLIMO [14], and Total Ownership Cost models for integrated systems, software and hardware [15]. He has advanced cost estimation for agile software processes [17] and GenAI usage [18] in systems and software processes. He is the developer of widely used online cost modeling tools, starting with Expert COCOMO, and including the current COCOMO [19] and COSYSMO tools [20], which are applied extensively across government, industry, and academia. He has codified these processes co-authoring the Project Planning chapter in the INCOSE SE Handbook [21].

The highly influential book *Software Cost Estimation with COCOMO II* [3], which he co-authored, established the standard cost model used globally across government and industry. He also co-edited the *Software Cost Estimation Metrics Manual for Defense Systems* [22], a key reference for the U.S. Department of Defense (DoD) for rigorous, data-driven cost estimation practices across acquisition programs. He was also key to improving software cost measurement policies in the DoD.

More recently, Madachy has focused on advancing systems engineering tool environments for digital engineering, as exemplified by his leadership on the open-source library se-lib [24] and his latest textbook *What Every Engineer Should Know About Python* [25].

He is at the forefront of leveraging and modeling generative AI processes in systems and software engineering, including cost and process modeling [18]. His recent tools [26], tutorials [27] [28] [29], keynote presentations [30], and panels led [31] reflect his leadership in using new technology to improve systems engineering processes. Throughout his career, integrating systems and software engineering processes has been a consistent thread [32], both in his own work and as a strategy to improve engineering disciplines.

System Dynamics and Systems Thinking

Madachy has made foundational contributions that apply systems thinking through system dynamics modeling. These began with his Ph.D. dissertation model to evaluate cost, schedule, and risk interdependencies in software projects. It established a framework that would underpin future research and applications across software, systems, and other domains.

Madachy's pioneering book, *Software Process Dynamics* [1], advanced the application of system dynamics modeling to simulate complex software development processes. It synthesizes research and practical experience into a self-contained, comprehensive resource for applying system dynamics. As INCOSE Pioneer and INCOSE Fellow Dr. Barry Boehm stated in the Foreword:

“The book brings together a tremendous amount of useful process modeling material and experience in using it in practical software decision situations. It organizes this material into a unifying framework that makes it easier to apply and explain, and illustrates it with a wide variety of useful examples. I believe that the book will serve as a standard reference for the software process dynamics field and a great help to practitioners and researchers for a good long time.”

Subsequently, he has applied system dynamics to evaluate new systems and software processes. These include hybrid, incremental processes for systems of systems in [5], which applied system dynamics to assess agile processes for coping with rapid change while simultaneously achieving high dependability. He has also used hybrid modeling with system dynamics and discrete-event simulation to study agile and lean systems engineering [10].

Beyond a major focus on systems and software processes, he has applied system dynamics to other system types. He addressed sustainability modeling in [6], where he led a team to model carbon emissions and transportation modes including port operations, rail, and truck logistics in a collaborative tool for evaluating tradeoffs. He applied system dynamics to space station issue resolution [8], modeling international collaboration dynamics to identify policy bottlenecks and resolution delays.

He has applied the methods to defense acquisition system modeling [9], which used system dynamics to highlight how acquisition feedback loops impact cost growth and program delays. He has also used system dynamics to assess

lawmaking processes [11] [12] to better understand law enactment and its feedback effects. This demonstrates systems thinking applied to societal-level processes.

His educational contributions augment his research. The textbook *What Every Engineer Should Know About Modeling and Simulation* [2] introduces system dynamics fundamentals (and simulation in general) to a broader engineering audience, making the methodology accessible and widely applicable across disciplines.

In order to lower the barriers to system modeling, he has developed comprehensive system dynamics capability in the open-source Systems Engineering Library (se-lib). The general-purpose system dynamics functions are comparable to those in Ithink/Stella and Vensim, and are interoperable with both. Se-lib supports larger digital engineering processes by combining system dynamics with discrete-event simulation, reliability modeling, SysML modeling, and other model types.

Overall, Madachy's work exemplifies system dynamics modeling as both a diagnostic and strategic tool for informing, strengthening, and unifying systems and software engineering practices.

Systems and Software Engineering Cost Modeling

While working in industry, Madachy was also a key member of the COCOMO II [3] development team which established the standard cost software model widely used across government and industry. He participated in the model development, project data collection from multiple companies used for the calibration, served as the primary COCOMO II trainer at organizations for the USC Center for Software Engineering, and authored large portions of the COCOMO II book.

He extended the COCOMO II model for the rule-based Expert COCOMO software project risk advisor and cost estimator [4] (also the first web-based COCOMO II tool), which later led to Expert COSYSMO [13] for systems engineering risk assessment.

Next he transitioned the academic research into cost estimation tools and processes used in industry as Chief Scientist at C-Bridge Internet Solutions, then Chief Science Officer for the Cost Xpert Group leading model research, tool development and training until the company was acquired.

After Cost Xpert Group he went into academia full-time as a Research Associate Professor at USC continuing cost modeling and system dynamics research. Funded research included developing dynamic cost and defect models for NASA Jet Propulsion Laboratories [7], and dynamic cost modeling of agile processes for Software-Intensive Systems of Systems for the Army [5]. He was serving as Interim Director of the USC Systems Architecting and Engineering program when NPS offered a tenure-track position in 2008.

At NPS, Madachy has led definitive research in software cost modeling for the Department of Defense (DoD), directly shaping cost estimation practices and DoD policy for software project measurements. His role in advancing rigorous, data-driven methodologies culminated in co-editing the Software Cost Estimation Metrics Manual for Defense Systems [22], a key reference for DoD program offices and analysts.

An impactful contribution was the development of core metrics definitions for software project data reporting that were standardized across the DoD. These empirically grounded definitions were formally adopted into DoD policy and implemented in the Software Resources Data Report (SRDR), as governed under DoDI 5000.73 [22]. This change significantly improved the consistency and quality of reported software development data across DoD programs, enabling more accurate and stable models for decision-making.

At NPS he has made major advancements in system product line cost/investment modeling with COPLIMO [14], and Total Ownership Cost models for integrated systems, software and hardware [15]. Over a dozen student capstones and theses have applied COPLIMO and COSYSMO across DoD programs to support informed acquisition decisions for system product line investments [23] [14] [16].

He is the developer of widely used online cost modeling tools. His COCOMO [19] and COSYSMO [20] tools are used for hundreds of valid user estimates daily. These web-based tools lower the barriers to parametric cost modeling and have tangible impact, being widely applied across government, industry, and academia. He has also developed online variants of the COPLIMO model, as well as the more extensive System Cost Model Suite, which integrates systems, software, and hardware cost models. He continuously maintains and updates these tools for the community.

He has advanced cost estimation for new processes including Lean Systems Engineering [10], agile software processes in the DoD [17], and GenAI usage in systems and software processes [18]. The enhanced DoD SRDR structure was instrumental in enabling the first empirical cost and schedule estimation models calibrated for agile software development in defense systems, developed by Madachy and colleagues. The work was also recognized with multiple best paper awards.

Another contribution to the community has been his effort to formalize and disseminate best practices in cost estimation and planning. He has done this by codifying key processes through authorship and contributions to

standard handbooks, co-authoring the Project Planning chapter in INCOSE Systems Engineering Handbook versions [21]. He has also contributed to relevant sections in the Systems Engineering Body of Knowledge (SEBoK) for systems engineering management, cost estimation, system affordability, and others. More recently he authored a section in the Digital Engineering (DE) Measurement Framework for system sizing used in cost models.

Systems Engineering Tools for Digital Engineering

More recently, Madachy has focused on advancing systems engineering tool environments for digital engineering. These efforts are exemplified by his leadership on the open-source Systems Engineering Library (se-lib) [24], authoring a new textbook, presenting tutorials on open-source system modeling, conducting research on leveraging Generative AI in systems engineering, and developing new tools including customized large language models (LLMs) for systems engineering.

To lower the barriers to system modeling, he initiated and developed se-lib, an open-source Python library that offers integrated capabilities for system dynamics, discrete event simulation, reliability modeling, SysML modeling and diagramming, and other model types. Its simulation functionality is extensive and comparable to established commercial tools. By unifying these modeling approaches within a single Python-based environment, se-lib supports broader digital engineering workflows and simplifies model integration, analysis, and reuse.

Recognizing Python's extensive open-source capabilities and its potential as a unifying platform for digital engineering, he authored a textbook applicable to all engineering disciplines, *What Every Engineer Should Know About Python* [25]. INCOSE Fellow Dr. Jon Wade wrote in the book's Foreword:

"I wholeheartedly endorse this book as an important contribution to engineering education and look forward to my students using it as a reference. Ultimately, *What Every Engineer Should Know About Python* does more than teach Python syntax; it equips engineers to harness Python's full potential across diverse fields.

Ray Madachy has provided a resource that will empower engineers to use Python effectively, not only to solve problems but also to design innovative systems for the future. I am excited to see how this book will empower the next generation of engineers to not only accelerate the OODA loop but to redefine the very boundaries of what is possible in engineering today."

He is at the forefront of leveraging and modeling generative AI processes in systems and software engineering, including applications in cost and process modeling [18]. He has developed new parametric cost models for the use of AI in systems and software engineering, as well as a systems engineering process maturity framework to help organizations assess, improve, and sustain the use of AI [18].

His recent tools [26], tutorials [27] [28] [29], keynote presentations [30], and panel moderation [31] reflect his leadership in applying emerging technologies to improve systems engineering processes. He is currently slated to serve as the SEBoK lead for integrating and harmonizing digital engineering with Model-Based Systems Engineering (MBSE) throughout the body of knowledge.

Selected Publications and Tools

- [1] R. Madachy. *Software Process Dynamics*. Wiley-IEEE Press, Hoboken, NJ, 2008.
- [2] R. Madachy and D. Houston. *What Every Engineer Should Know About Modeling and Simulation*. CRC Press, Boca Raton, FL, 2017.
- [3] B. Boehm, C. Abts, W. Brown, S. Chulani, B. Clark, E. Horowitz, R. Madachy, D. Reifer, and B. Steece. *Software Cost Estimation with COCOMO II*. Prentice-Hall, 2000.
- [4] R. Madachy. Heuristic Risk Assessment Using Cost Factors. *IEEE Software*, 1997.
- [5] R. Madachy, B. Boehm, and J. Lane. Assessing Hybrid Incremental Processes for SISOS Development. *Software Process Improvement and Practice*, 2007.
- [6] R. Madachy, B. Haas, H. Bradbury-Huang, J. Newell, R. Vos, and M. Rahimi. Achieving sustainable development in southern california: Collaborative learning through system dynamics modeling. In *Proceedings of the 18th International Symposium of INCOSE*, Netherlands, 2008.
- [7] R. Madachy and B. Boehm. Assessing quality processes with ODC COQUALMO. In *Proceedings of the 2008 International Conference on Software Process*, Liepzig, Germany, 2008.

- [8] H. Johnson and R. Madachy. Simulating international space station issue resolution. In *Proceedings of AIAA Space 2008 Conference*, Long Beach, CA, 2008.
- [9] J. Wirthlin, D. Houston, and R. Madachy. Defense acquisition system simulation studies. In *Proceedings of the 2011 International Conference on Software and System Process*, Waikiki, HI, 2011. IEEE Computer Society.
- [10] R. Turner, R. Madachy, D. Ingold, and J. Lane. Modeling kanban processes in systems engineering. In *Proceedings of the 2012 International Conference on Software and System Process*, Zurich, Switzerland, 2012. IEEE.
- [11] R. Madachy. System dynamics structures for modeling lawmaking processes. In *Proceedings of the 3rd Annual Science of Laws Conference*. The Science of Laws Institute, 2016.
- [12] R. Madachy. System dynamics behaviors for modeling lawmaking processes. In *Proceedings of the 4th Annual Science of Laws Conference*. The Science of Laws Institute, November 2017.
- [13] R. Madachy, and R. Valerdi. Expert COSYSMO Systems Engineering Cost Model and Risk Advisor. In N. Scala and J. Howard, editors, *Handbook of Military and Defense Operations Research*. CRC Press, Boca Raton, FL, 2020.
- [14] R. Madachy and J.M. Green. System Product Line Cost and Investment Modeling Applied to UUVs. In *Proceedings of the 21st Annual Acquisition Research Symposium*. Acquisition Research Program, 2024.
- [15] R. Madachy. System Cost Model Suite v.2.0, 2017 – 2022. URL https://softwarecost.org/tools/cost_model_suite/.
- [16] R. Madachy, and J. Green. An MBSE Approach for System Product Line Architecture Modeling and Economic Analysis. In *Proceedings of the 2018 INCOSE San Diego Mini-Conference*, San Diego, CA, December 2018.
- [17] Rosa, W., Clark, B., Madachy, R. & Boehm, B. Empirical Effort and Schedule Estimation Models for Agile Processes in the US DoD. *IEEE Transactions On Software Engineering*. 48(8), 2022.
- [18] R. Madachy, R. Bell, and R. Longshore. A Generative AI-Driven Systems Engineering Maturity and Cost Modeling Framework. In *Proceedings of the 2025 Conference on Systems Engineering Research*. Procedia Computer Science, 2025.
- [19] R. Madachy. COCOMO II, 2006 – 2023. URL <https://softwarecost.org/tools/COCOMO>.
- [20] R. Madachy. COSYSMO, 2011 – 2024. URL <http://softwarecost.org/tools/COSYSMO>.
- [21] International Council on Systems Engineering (INCOSE). *"Project Planning" in Systems Engineering Handbook: A Guide for System Life Cycle Processes and Activities, Version 5.0*. John Wiley & Sons, Hoboken, NJ, 2023.
- [22] B. Clark and R. Madachy, editors. *Software Cost Estimation Metrics Manual for Defense Systems*. Software Metrics Inc., Haymarket, VA, 2015.
- [23] R. Madachy, and J.M. Green. Systems and Cost Effectiveness Modeling of Unmanned Systems Product Lines for Acquisition. In *Proceedings of the 19th Annual Acquisition Research Symposium*, Acquisition Research Program, 2022.
- [24] R. Madachy, and others. se-lib: Systems Engineering Library (Python package), 2023–2025. URL <https://pypi.org/project/se-lib/>, <https://github.com/se-lib/se-lib>.
- [25] R. Madachy. *What Every Engineer Should Know About Python*. CRC Press, Boca Raton, FL, 2025.
- [26] R. Madachy. COSYSMO with AI Usage, 2025. URL https://softwarecost.org/tools/COSYSMO_AI/.
- [27] R. Madachy, R. Longshore, and R. Bell. Tutorial: Open Source System Modeling with Python and Generative AI. In *Proceedings of the 35th International Symposium of INCOSE*, Ottawa, Canada, 2025.

- [28] R. Madachy and R. Bell. Tutorial: Developing Custom LLMs for Systems Engineering. In *Proceedings of the 35th International Symposium of INCOSE*, Ottawa, Canada, 2025.
- [29] R. Madachy and R. Longshore. Tutorial: Open Source System Modeling with Python. In *Proceedings of the 34th International Symposium of INCOSE*, Dublin, Ireland, 2024.
- [30] R. Madachy. Keynote: Open Source Systems Modeling and Convergence with AI. In *Proceedings of the 2024 Boehm Center for Systems and Engineering Annual Research Review*, Virtual, April 2024.
- [31] R. Madachy, R. Valerdi, G. Wang, B. Brown, and M. Wheaton. Panel: Cost Impacts of Generative AI in Systems Engineering Processes (Moderated by R. Madachy). In *Proceedings of the 35th International Symposium of INCOSE*, Ottawa, Canada, 2025.
- [32] R. Fairley, T. Hilburn, R. Madachy, and A. Squires. "Systems Engineering and Software Engineering", in *SEBoK Editorial Board. 2024. The Guide to the Systems Engineering Body of Knowledge (SEBoK), v. 2.12, N. Hutchison (Editor in Chief)*. Hoboken, NJ: The Trustees of the Stevens Institute of Technology 2024. URL https://sebokwiki.org/wiki/Systems_Engineering_and_Software_Engineering.

Appendix of Book Forewords

Software Process Dynamics Foreword by INCOSE Pioneer and INCOSE Fellow Dr. Barry Boehm

FOREWORD

The pace of change in software-intensive systems continues to accelerate at a dizzying rate. This presents a huge challenge for people trying to develop useful software. In the early days of software development, developers could freeze the requirements for the software, develop the software to the requirements, and deliver the resulting software two years later with confidence that the requirements would still be relevant and the software would be useful. Most of our software engineering processes, methods, and tools were developed and used under the assumption of relatively stable requirements. Examples are formal specification languages, performance-optimized point-solution designs, fixed-requirements software-cost estimation, earned-value management systems, requirements traceability matrices, fixed-price/fixed-requirements contracts, and a general attitude that “requirements creep” was bad in that it destabilized software development.

However, as these practices became increasingly institutionalized, the accelerating rate of software change made them increasingly risky to use. Projects would use them for two years and become extremely frustrated when the users were not interested in the obsolete capabilities that resulted. Projects would fall behind schedule and use static models (time to complete = work remaining divided by work rate) to try to make up time by adding people, and run afoul of Brooks’s law (adding people to a late software project will make it later). Or they would sprint for the finish line using a point-solution design that satisfied the initial requirements but was extremely difficult to modify when trying to satisfy users’ changing requirements. Ironically, even with all of these difficulties, organizations would increasingly turn to software and its ability to be electronically upgraded as their best way to adapt their products, services, and systems to the increasing pace of change in their business or operational environment.

In order to keep up with this increased demand for software and the rapid pace of change, software organizations and projects need better ways to reason about the effects of change on their software products, projects, and processes. This is often very difficult to do, as software changes often have complex second-order and higher-order interaction effects that are hard to visualize and reason about. Thus, for example, a project with high rates of change in its requirements, being developed by a team with high rates of change in its personnel, will need to understand and control the interactions of decreased productivity due to change processing, decreased productivity due to new staff members’ unfamiliarity with the project and product, and loss of productivity when key staff members are diverted from project work to train and mentor new people in order to develop increased downstream productivity.

One of the best techniques for reasoning about the effects of such complex interacting changes is the System Dynamics modeling framework that Ray Madachy presents in this book. As I have found in numerous applications of the method, it enables project personnel to model such effects and run the models to better understand the implications of candidate project strategies and decisions.

From the pioneering application of Jay Forrester’s System Dynamics approach to software project modeling by Tarek Abdel-Hamid and Stuart Madnick in their 1991 book *Software Project Dynamics*, system dynamics modeling has been applied to many aspects of software development and management. These include the analysis of the effects on software system cost, schedule, and quality of alternative approaches to software process sequencing, software requirements determination, software architecture and design, software project organization and staffing, software development and integration, software quality assurance, and software change management. These applications have constituted the major source of solution approaches in the software process simulation community and its annual series of ProSim conferences (recently combined with the Software Process Workshop into the International Conference on Software Process, held concurrently with the International Conference on Software Engineering).

Ray Madachy has been a major contributor to this body of work. His experience in applying system dynamics modeling as a technical leader in such diverse organizations as the aerospace corporation Litton Systems, the e-commerce system development company C-Bridge Institute, and the software tools company Cost Xpert Group have given him a broad and deep perspective on the critical success factors of developing and applying system dynamics models to various classes of software decision situations. His experience in teaching and researching these techniques at USC has enabled him to develop an integrating framework and set of techniques that make system dynamics modeling much easier and cost-effective to learn and apply to a software decision situation. His resulting book begins with an overview of the systems dynamics modeling framework, and an example application showing how to develop a system dynamics model that helps explain the conditions under which you can add people to

a software project with or without having Brooks's Law apply. Next is an extensive chapter on the modeling process that introduces concepts and techniques used to develop system dynamics models, with illustrations of how they are developed and applied. The next chapter provides and explains a full range of model structures from modeling molecules to large infrastructures and flow chains that can be used in new models. Once this framework is established, there are three chapters that apply it to the most significant classes of system dynamics applications.

The first of these chapters covers useful applications to software personnel decisions: workforce levels and team composition, learning, burnout, skills, motivation, retention, and rotation effects. The second chapter covers software process and product decision situations: peer reviews, software reuse, COTS-based system development, software architecting, and software process improvement effects. The third chapter covers project and organization applications, such as business case analysis, defect reduction strategies, and staffing management strategies. The final chapter projects likely future uses and research challenges in software process modeling and simulation, including how system dynamics models can add value via integration with other classes of models.

The appendices also provide important and one-of-a-kind material. The first appendix covers statistics of simulation, which is not covered in traditional references on system dynamics. Next is a thorough annotated bibliography of work using system dynamics for software processes, and is the definitive compendium for the field. Finally, the last appendix lists executable models provided with the book. These are used in examples and can be used for further exercises or for incorporation into your own models. These are valuable, reusable, and fun assets to go along with the book.

Overall, the book brings together a tremendous amount of useful process modeling material and experience in using it in practical software decision situations. It organizes this material into a unifying framework that makes it easier to apply and explain, and illustrates it with a wide variety of useful examples. I believe that the book will serve as a standard reference for the software process dynamics field and a great help to practitioners and researchers for a good long time.

BARRY BOEHM
Los Angeles, California
University of Southern California

***What Every Engineer Should Know About Python* Foreword by IN-COSE Fellow Dr. Jon Wade**

As an engineer with over 25 years in industry and 15 years in academia, spanning a wide range of domains from supercomputing to complex enterprise systems, I've had a front-row seat to the evolution of technology and the growing importance of data and computation in driving informed, adaptive engineering practices. Entering Industry 5.0, we see data as a powerful catalyst that accelerates the OODA loop (observe, orient, decide, and act) in real-time—reshaping not only how we understand the world but also how we design, troubleshoot, and innovate.

Python has emerged as the common language of this data-driven revolution, providing engineers with a flexible, high-level programming environment that is both intuitive and immensely powerful. It is fair to say Python has become the "Esperanto" of data sciences, AI, and machine learning, empowering professionals across disciplines to speak a common language, share solutions, and drive advancements in ways previously unimaginable. For engineers, especially, Python is no longer just a coding skill but a central tool in our arsenal.

What Every Engineer Should Know About Python is a timely and invaluable resource, precisely because it balances the essential principles of Python with its real-world application across various engineering practices. It equips engineers with not only the syntax and structure of Python but also the robust utility libraries, scientific tools, and engineering-specific applications that can accelerate and enhance our work.

This book serves as a complete reference for engineers to find everything in one place with explicit examples. It begins with the basics, guiding readers from foundational Python concepts to more advanced programming techniques, including the use of powerful libraries like NumPy, Matplotlib, and Pandas. These chapters ensure that every engineer, regardless of their familiarity with Python, can quickly become competent and confident in using it. Further, the book covers complex topics like object-oriented programming, embedded systems, and even cutting-edge applications in machine learning, robotics, and web-based interfaces.

In addition to providing hands-on examples like projectile motion and wind turbine power simulation, this book also takes a structured dive into Python applications for critical engineering analysis—from statistical studies to simulations of robotic paths and manufacturing processes. Each chapter is carefully designed to build practical skills that bridge theoretical concepts with their real-world engineering counterparts.

As we move towards a future where young engineers will increasingly work on transdisciplinary projects, this book also introduces modeling notations from systems and software engineering to facilitate cross-discipline communication. This forward-looking approach will undoubtedly help engineers adapt to evolving industry demands.

I wholeheartedly endorse this book as an important contribution to engineering education and look forward to my students using it as a reference. Ultimately, *What Every Engineer Should Know About Python* does more than teach Python syntax; it equips engineers to harness Python's full potential across diverse fields.

Ray Madachy has provided a resource that will empower engineers to use Python effectively, not only to solve problems but also to design innovative systems for the future. I am excited to see how this book will empower the next generation of engineers to not only accelerate the OODA loop but to redefine the very boundaries of what is possible in engineering today.

Dr. Jon Wade

Professor of Practice, Director of Convergent Systems Engineering, Jacobs School of Engineering, University of California, San Diego; Fellow, International Council of Systems Engineering (INCOSE)